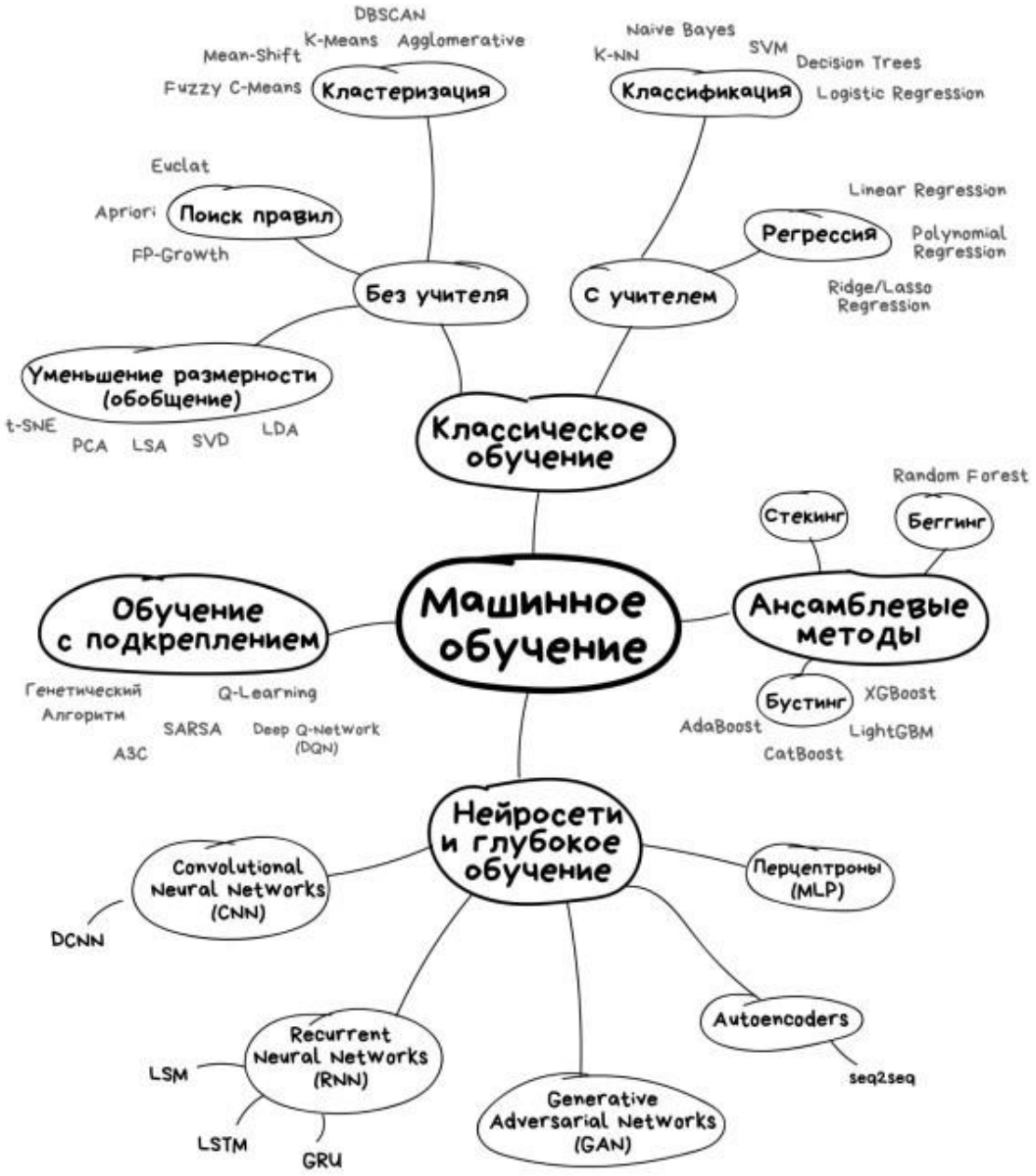
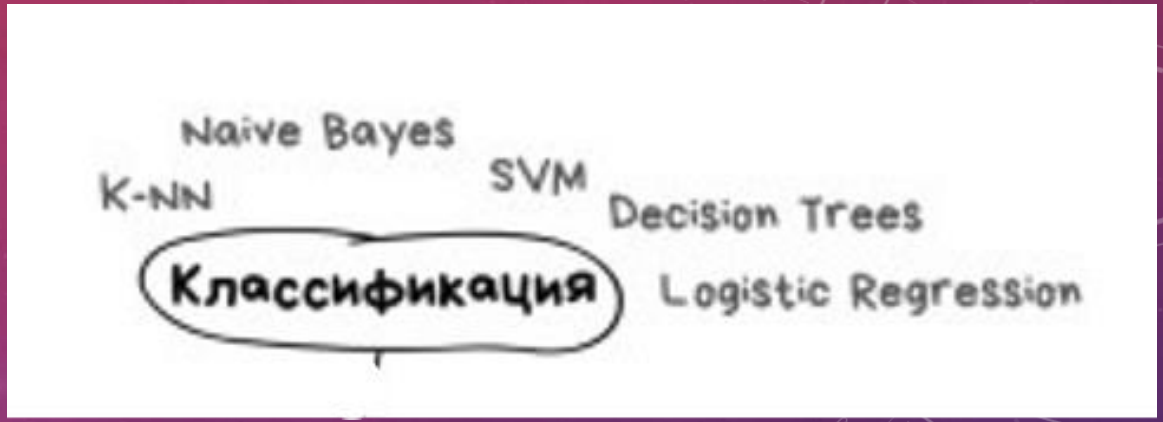
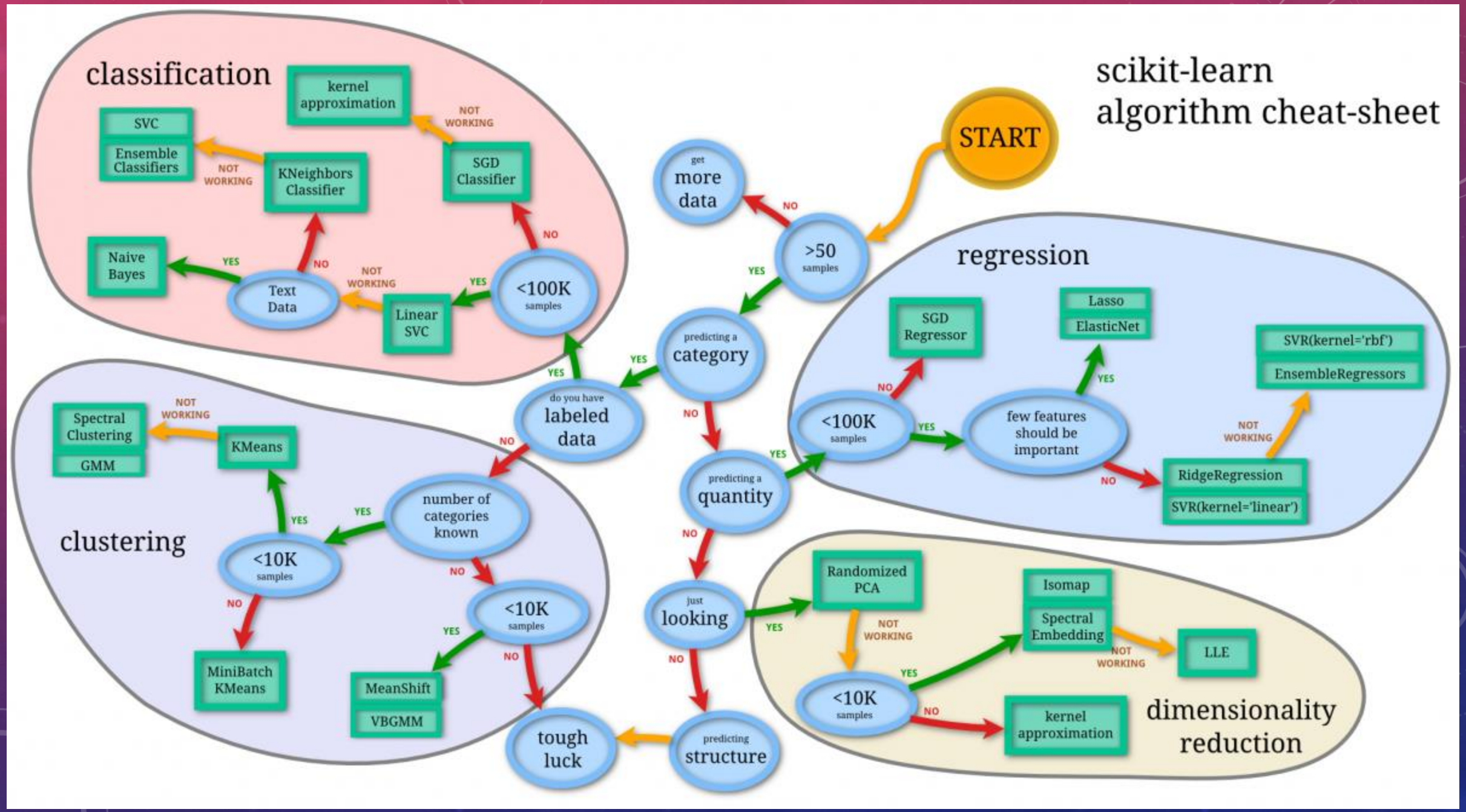
МЕТОДЫ МАШИННОГО ОБУЧЕНИЯ С УЧИТЕЛЕМ

КЛАССИФИКАЦИЯ


DBSCAN
K-Means
Agglomerative
Mean-Shift
Fuzzy C-Means
Кластеризация
Naive Bayes
K-NN
SVM
Decision Trees
Классификация
Logistic Regression
Euclat
Apriori
Поиск правил
FP-Growth
Linear Regression
Регрессия
Polynomial Regression
Ridge/Lasso Regression
Без учителя
С учителем
Уменьшение размерности (обобщение)
t-SNE
PCA
LSA
SVD
LDA
Классическое обучение
Random Forest
Стекинг
Беггинг
Обучение с подкреплением
Машинное обучение
Ансамблевые методы
Генетический Алгоритм
Q-Learning
SARSA
Deep Q-Network (DQN)
A3C
Бустинг
XGBoost
AdaBoost
LightGBM
CatBoost
Нейросети и глубокое обучение
Convolutional Neural Networks (CNN)
DCNN
Перцептроны (MLP)
Autoencoders
seq2seq
Recurrent Neural Networks (RNN)
LSM
LSTM
GRU
Generative Adversarial Networks (GAN)


Naive Bayes
K-NN
SVM
Decision Trees
Классификация
Logistic Regression


scikit-learn
algorithm cheat-sheet
START
>50
samples
NO
get
more
data
YES
predicting a
category
YES
do you have
labeled
data
YES
<100K
samples
NO
SGD
Classifier
NOT
WORKING
kernel
approximation
classification
<100K
samples
YES
Linear
SVC
NOT
WORKING
Text
Data
NO
KNeighbors
Classifier
NOT
WORKING
SVC
Ensemble
Classifiers
YES
Naive
Bayes
NO
predicting a
quantity
YES
<100K
samples
NO
SGD
Regressor
YES
few features
should be
important
YES
Lasso
ElasticNet
NO
RidgeRegression
SVR(kernel='linear')
NOT
WORKING
SVR(kernel='rbf')
EnsembleRegressors
regression
NO
number of
categories
known
YES
<10K
samples
YES
KMeans
NOT
WORKING
Spectral
Clustering
GMM
NO
MiniBatch
KMeans
NO
<10K
samples
YES
MeanShift
VBGMM
NO
clustering
NO
just
looking
YES
Randomized
PCA
NOT
WORKING
<10K
samples
YES
Isomap
Spectral
Embedding
NOT
WORKING
LLE
NO
kernel
approximation
dimensionality
reduction
NO
predicting
structure
tough
luck

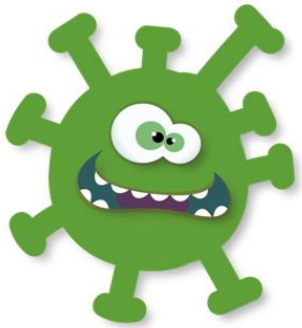
КЛАССИФИКАЦИЯ

- **Задача классификации в машинном обучении** — это задача отнесения объекта к одному из заранее определенных классов на основании его формализованных признаков. Каждый из объектов в этой задаче представляется в виде вектора в N-мерном пространстве, каждое измерение в котором представляет собой описание одного из признаков объекта. Для обучения классификатора необходимо иметь набор объектов, для которых заранее определены классы. Это множество называется **обучающей выборкой**, её разметка производится вручную, с привлечением специалистов в исследуемой области
- Задача классификации (как, впрочем и следует из названия) сводится к отнесению конкретного объекта к одному из заранее известных классов. Вот эта “метка” - название класса - и выступает в роли **целевой переменной**. И, совершенно естественно, что классов может быть только конечное количество, поэтому целевая переменная - дискретная.

ПРИМЕРЫ БИНАРНОЙ КЛАССИФИКАЦИИ

ИЛИ

ИЛИ



ИЛИ

АЛГОРИТМЫ КЛАССИФИКАЦИИ

Scikit-Learn обеспечивает легкий доступ к многочисленным различным алгоритмам классификации.

К числу таких классификаторов относятся:

- К-Ближайшие Соседи
- Машины опорных Векторов
- Классификаторы дерева решений / Случайные леса
- Наивный Байес
- Линейный дискриминантный анализ
- Логистическая регрессия

МЕТОД БЛИЖАЙШИХ СОСЕДЕЙ (K NEAREST NEIGHBORS)

Все объекты расположены в многомерном пространстве

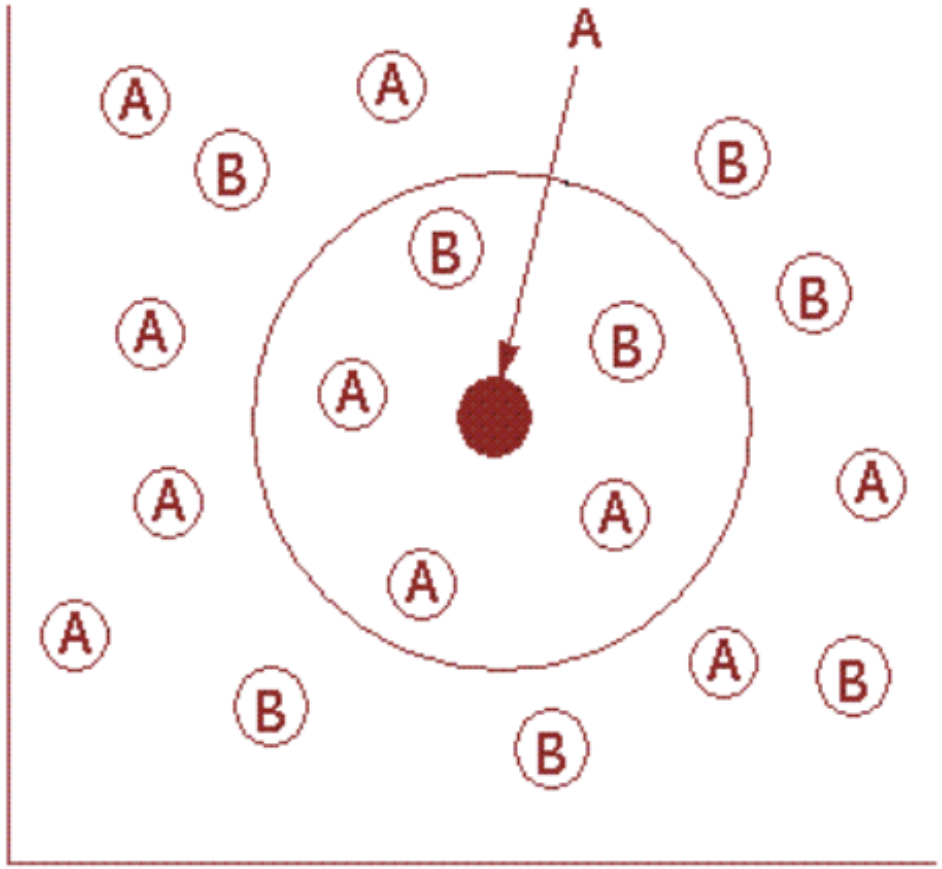
1. Задаем число k – количество ближайших соседей.

2. Ищем k объектов с минимальным расстоянием до нашего нового объекта. Используем меру для расчета расстояний.

3.1 Простое невзвешенное голосование. Считаем сколько объектов с классами присутствует внутри заданного расстояния. Например, если число объектов с классом А большинство, то новый объект относится к классу А.

3.2. Взвешенное голосование

В такой ситуации учитывается также и расстояние до новой записи. Чем меньше расстояние, тем более значимый вклад вносит голос.



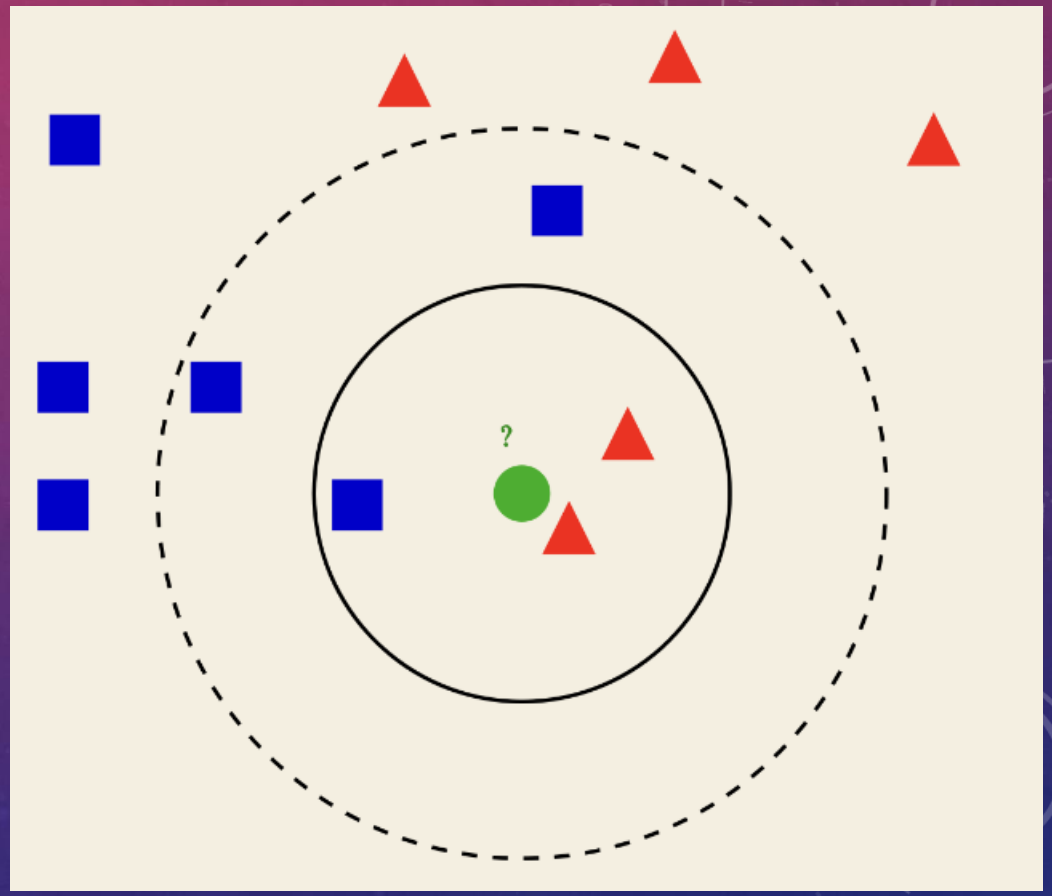
МЕТОД К-БЛИЖАЙШИХ СОСЕДЕЙ

Преимущества и недостатки

- Этот алгоритм довольно прост в своей реализации и устойчив к зашумленным обучающим данным. Даже если обучающие данные велики, они достаточно эффективны. Единственным недостатком алгоритма KNN является то, что нет необходимости определять значение K, а вычислительные затраты довольно высоки по сравнению с другими алгоритмами.

Примеры использования

- Промышленное применение для поиска аналогичных задач по сравнению с другими
- Приложения для обнаружения рукописного ввода
- Распознавание изображений
- Распознавание видео
- Анализ запасов



МЕТОД К-БЛИЖАЙШИХ СОСЕДЕЙ

Некоторые преимущества метода KNN:

- **Простота**. Алгоритм не требует сложных математических вычислений и может быть реализован с минимальными усилиями.
- **Отсутствие обучения**. Нет необходимости в обучении модели, что экономит время и ресурсы.
- **Гибкость**. Может использоваться для классификации и регрессии.

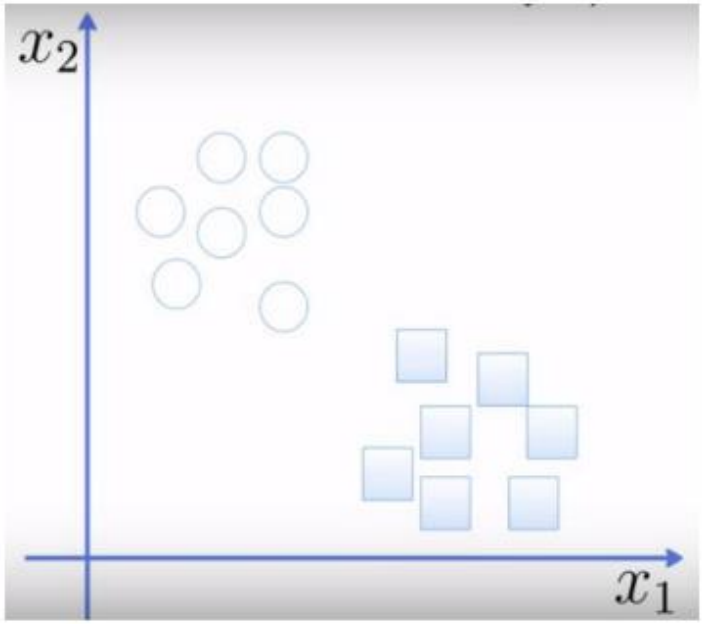
Некоторые недостатки метода KNN:

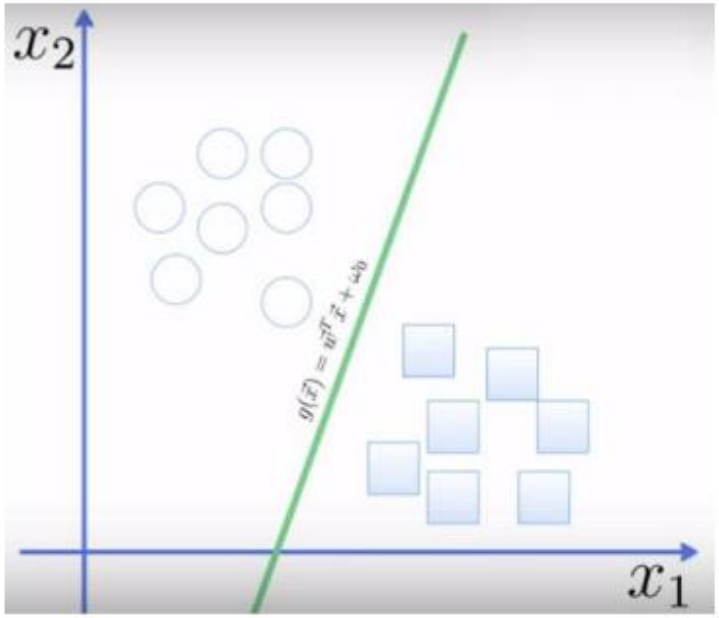
- **Высокие вычислительные затраты**. Для больших наборов данных вычисление расстояний может быть очень затратным.
- **Чувствительность к шуму**. KNN чувствителен к шуму и выбросам в данных.
- **Выбор К**. Оптимальное значение К может быть трудно определить.

МЕТОД ОПОРНЫХ ВЕКТОРОВ (SVM)

Каждый объект данных (например, документ, котировки ценных бумаг или компании) представлен как вектор в **P** мерном пространстве (последовательность чисел). Пусть у нас есть тестовая коллекция, в которой есть набор объектов (features) и есть набор классов. Математическая задача обучения заключается в том что бы найти функцию, которая адекватно сопоставляла объекты и классы, то есть найти такую функцию, которая эффективно разделяла бы объекты в пространстве features.

Рассмотрим пример на плоскости: У нас есть два класса с двумя features (x1, x2). Нужной найти прямую линию, которая оптимально разделяла два класса.



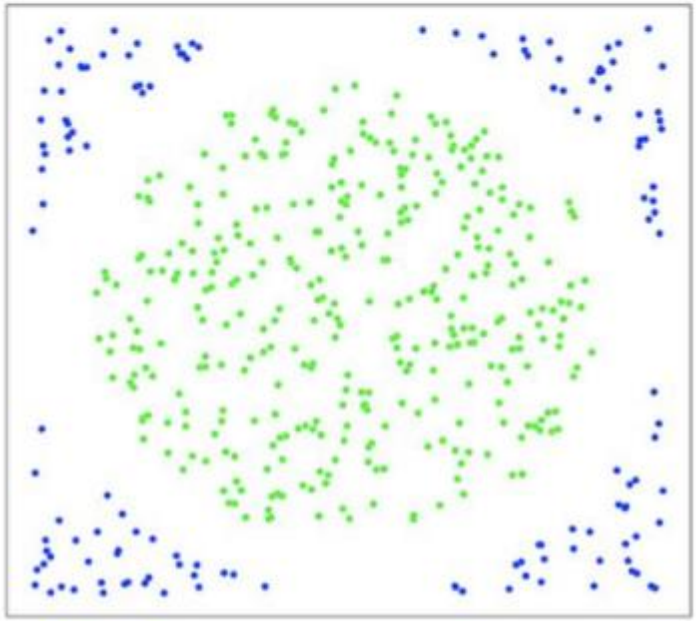


МЕТОД ОПОРНЫХ ВЕКТОРОВ (SVM)

Нахождение уравнения плоскости является стандартной задачей квадратичного программирования и решается с помощью множителей Лагранжа. Собственно в этом заключается процесс обучения.

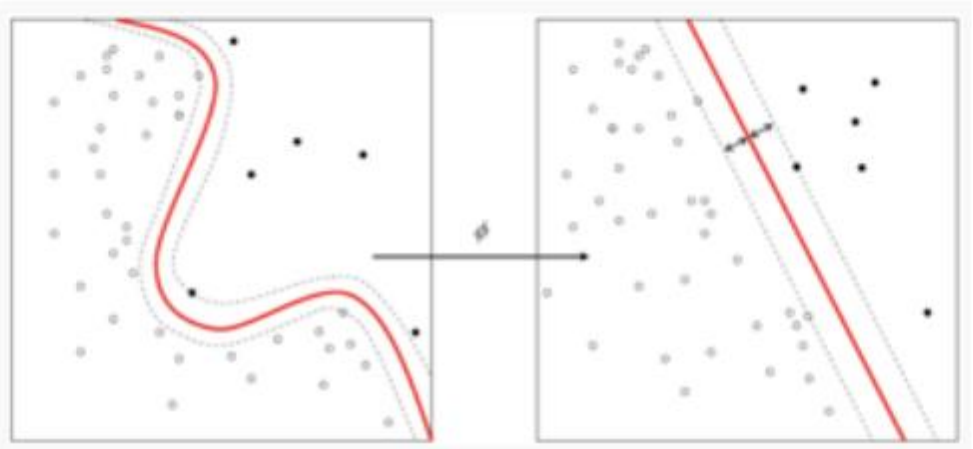
Как только плоскость найдена, берем новый объект и смотрим где он расположен относительно плоскости. Если справа, то принадлежит одному классу, если слева, то наш объект принадлежит другому классу.

Однако, ка правило на практике встречаются случаи когда объекты расположены, так что на плоскости невозможно провести разделяющую прямую. В этом случае плоскость вкладывается в пространство большей размерности. При вложении плоскость трансформируется таким образом, что бы появилась возможность провести разделяющую плоскость.



Демонстрация подобного преобразования

https://youtu.be/3liCbRZPrZA



Плюсы и минусы классического SVM:

Преимущества:

- хорошо работает с пространством признаков большого размера;
- хорошо работает с данными небольшого объема;
- алгоритм максимизирует разделяющую полосу, которая, как подушка безопасности, позволяет уменьшить количество ошибок классификации;
- так как алгоритм сводится к решению задачи квадратичного программирования в выпуклой области, то такая задача всегда имеет единственное решение (разделяющая гиперплоскость с определенными гиперпараметрами алгоритма всегда одна).

Недостатки:

- долгое время обучения (для больших наборов данных);
- неустойчивость к шуму: выбросы в обучающих данных становятся опорными объектами-нарушителями и напрямую влияют на построение разделяющей гиперплоскости;
- не описаны общие методы построения ядер и спрямляющих пространств, наиболее подходящих для конкретной задачи в случае линейной неразделимости классов. Подбирать полезные преобразования данных – искусство.

КЛАССИФИКАТОР ДЕРЕВА РЕШЕНИЙ (DECISION TREE CLASSIFIER)

- Этот классификатор разбивает данные на всё меньшие и меньшие подмножества на основе разных критериев, т. е. у каждого подмножества своя сортирующая категория. С каждым разделением количество объектов определённого критерия уменьшается.
- Классификация подойдёт к концу, когда сеть дойдёт до подмножества только с одним объектом. Если объединить несколько подобных деревьев решений, то получится так называемый *Случайный Лес* (англ. *Random Forest*).

КЛАССИФИКАТОР ДЕРЕВА РЕШЕНИЙ (DECISION TREE CLASSIFIER)

Преимущества

- простота и интерпретируемость, требует очень небольшой подготовки данных
- может работать с данными различных типов (категориальные, бинарные, числовые)
- может работать с несбалансированными данными
- может использоваться для отбора признаков

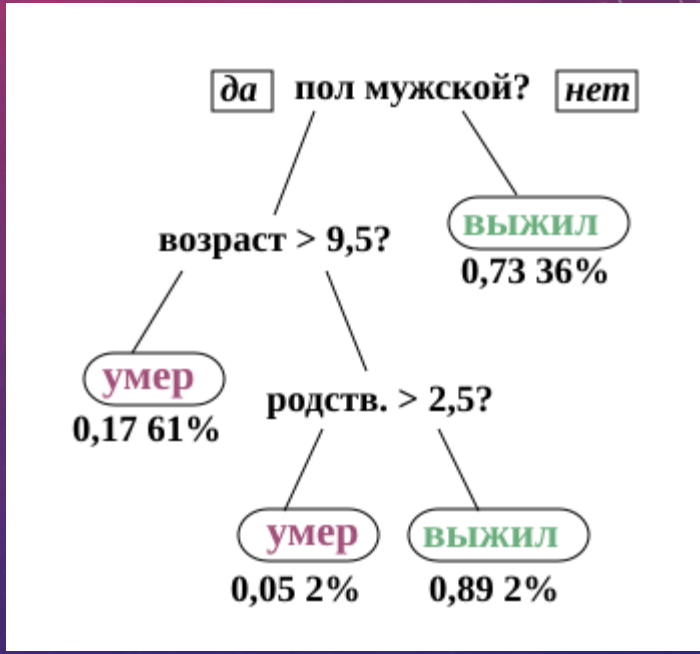
Некоторые недостатки:

- склонен к переобучению (overfitting) при большой глубине дерева или при отсутствии ограничений на минимальное количество объектов в листе
- неустойчив к шуму в данных
- не гарантирует наилучшее решение, так как выбор оптимального разделения происходит на каждом шаге по отдельности, а не глобально

КЛАССИФИКАТОР ДЕРЕВА РЕШЕНИЙ (DECISION TREE CLASSIFIER)

Примеры использования

- Исследование данных
- Распознавание образов
- Ценообразование опционов в финансах
- Выявление угроз заболеваний и рисков



НАИВНЫЙ БАЙЕСОВСКИЙ КЛАССИФИКАТОР (NAIVE BAYES)

Формула Байеса:

$$P(A|B) = \frac{P(B|A)\,P(A)}{P(B)}.$$

- Наивный байесовский классификатор — вероятностный классификатор на основе формулы Байеса со строгим (наивным) предположением о независимости признаков между собой при заданном классе.
- Такой классификатор вычисляет вероятность принадлежности объекта к какому-то классу. Эта вероятность вычисляется из шанса, что какое-то событие произойдёт, с опорой на уже на произошедшие события.
- Каждый параметр классифицируемого объекта считается независимым от других параметров.

НАИВНЫЙ БАЙЕСОВСКИЙ КЛАССИФИКАТОР (NAIVE BAYES)

Преимущества и недостатки

- Наивный байесовский классификатор требует небольшого объема обучающих данных для оценки необходимых параметров для получения результатов. Они чрезвычайно быстры по своей природе по сравнению с другими классификаторами.
- Единственным недостатком является то, что они, как известно, являются плохими оценщиками.

Примеры использования

- Предсказания заболеваний
- Классификация документов
- Спам-фильтры
- Анализ настроений

ЛИНЕЙНЫЙ ДИСКРИМИНАНТНЫЙ АНАЛИЗ (LINEAR DISCRIMINANT ANALYSIS)

- Этот метод работает путём уменьшения размерности набора данных, проецируя все точки данных на линию. Потом он комбинирует эти точки в классы, базируясь на их расстоянии от центральной точки.
- Этот метод, как можно уже догадаться, относится к линейным алгоритмам классификации, т. е. он хорошо подходит для данных с линейной зависимостью.
- Объекты разных классов должны иметь нормальное распределение и располагаться как можно дальше друг от друга, а одного класса — как можно ближе.

ЛОГИСТИЧЕСКАЯ РЕГРЕССИЯ В ЗАДАЧАХ КЛАССИФИКАЦИИ

Логистическая регрессия – это разновидность множественной регрессии, общее назначение которой состоит в анализе связи между несколькими независимыми переменными (называемыми также регрессорами или предикторами) и зависимой переменной. Бинарная логистическая регрессия, как следует из названия, применяется в случае, когда зависимая переменная является бинарной (т.е. может принимать только два значения). Иными словами, с помощью логистической регрессии можно оценивать вероятность того, что событие наступит для конкретного испытуемого (больной/здоровый, возврат кредита/дефолт и т.д.).

$$y = a + b_1 x_1 + b_2 x_2 + \ldots + b_n x_n$$

Для решения проблемы задача регрессии может быть сформулирована иначе: вместо предсказания бинарной переменной, мы предсказываем непрерывную переменную со значениями на отрезке [0,1] при любых значениях независимых переменных. Это достигается применением следующего регрессионного уравнения (логит-преобразование):

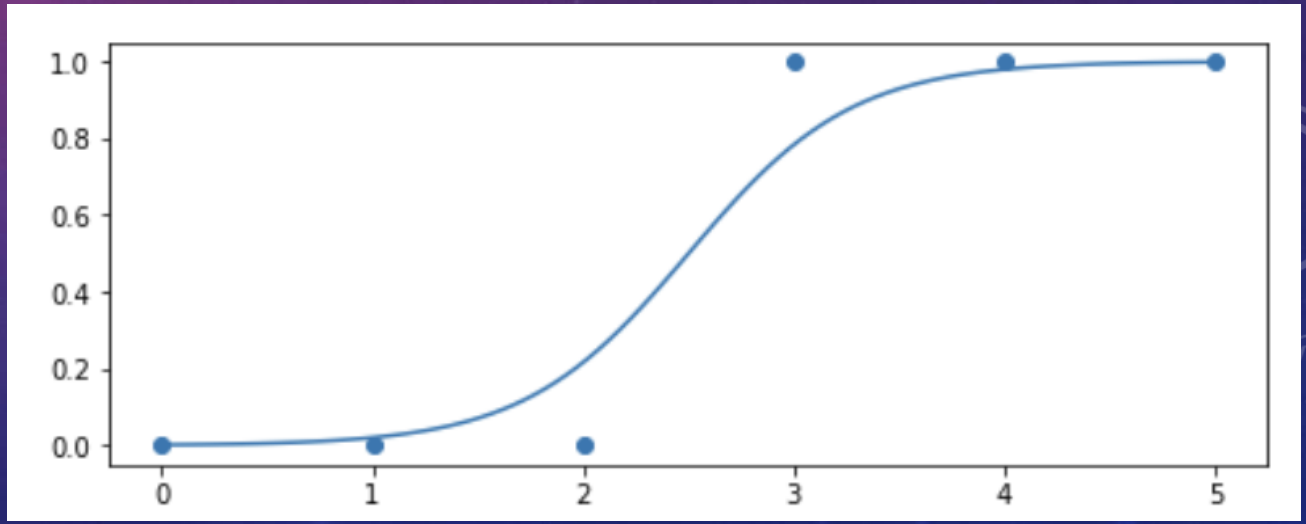
$$p = \frac{1}{1+e^{-y}},$$

где P – вероятность того, что произойдет интересующее событие; e – основание натуральных логарифмов 2,71…; y – стандартное уравнение регрессии.

ЛОГИСТИЧЕСКАЯ РЕГРЕССИЯ

- Логистическая регрессия выводит прогнозы о точках в бинарном масштабе — нулевом или единичном. Если значение чего-либо равно либо больше 0.5, то объект классифицируется в большую сторону (к единице). Если значение меньше 0.5 — в меньшую (к нулю).

- У каждого признака есть своя метка, равная только 0 или только 1. Логистическая регрессия является линейным классификатором и поэтому используется, когда в данных прослеживается какая-то линейная зависимость.



ЛОГИСТИЧЕСКАЯ РЕГРЕССИЯ

Преимущества и недостатки

- Логистическая регрессия специально предназначена для классификации, она полезна для понимания того, как набор независимых переменных влияет на результат зависимой переменной.
- Основным недостатком алгоритма логистической регрессии является то, что он работает только тогда, когда прогнозируемая переменная является двоичной, он предполагает, что в данных нет пропущенных значений, и предполагает, что предикторы независимы друг от друга.

Примеры использования

- Выявление факторов риска заболеваний
- Классификация слов
- Прогноз погоды
- Заявки на голосование

Меры качества классификаторов для бинарных классов

$$\text{Precision} = \frac{\text{TP}}{TP + FP}$$

Precision: число правильно предсказанных положительных значений поделенных на число предсказанных классификатором положительных значений.

$$\text{Recall} = \frac{\text{TP}}{TP + FN}$$

Recall : число правильно предсказанных положительных значений поделенных на число положительных ответов в данных.

	Predicted 1	Predicted 0
True 1	true positive	false negative
True 0	false positive	true negative

	Predicted 1	Predicted 0
True 1	TP	FN
True 0	FP	TN

	Predicted 1	Predicted 0
True 1	hits	misses
True 0	false alarms	correct rejections

	Predicted 1	Predicted 0
True 1	P(pr1|tr1)	P(pr0|tr1)
True 0	P(pr1|tr0)	P(pr0|tr0)

Confusion Matrix

TP – число правильно предсказанных положительных значений

FN – число неправильно предсказанных положительных значений

FN – число правильно предсказанных негативных значений

FP – число неправильно предсказанных негативных значений

Меры качества классификаторов для многих классов

$$\text{Precision} = \frac{\sum_{i=1}^{l} tp_i}{\sum_{i=1}^{l} (tp_i + fp_i)}$$

$$\text{Recall} = \frac{\sum_{i=1}^{l} tp_i}{\sum_{i=1}^{l} (tp_i + fn_i)}$$

$$\text{F measure} = \frac{(\beta^2+1)\,Precision_\mu Recall_\mu}{\beta^2 Precision_\mu + Recall_\mu}$$

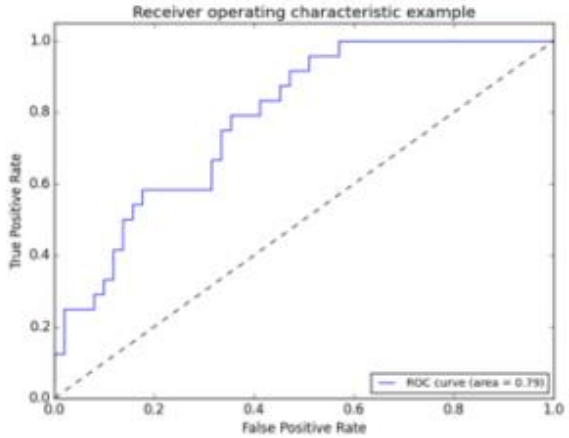
МЕРЫ КАЧЕСТВА КЛАССИФИКАТОРОВ ДЛЯ БИНАРНЫХ КЛАССОВ

F – measure

$$F - measure = \frac{(\beta^2 + 1) * precision * recall}{\beta^2 * precision + recall} \Longleftrightarrow \frac{(\beta^2+1)tp}{(\beta^2+1)tp+\beta^2 fn+fp}$$

β– обычно берут равной 1. F measure = 2 * (precision * recall) / (precision + recall)

The F measure (F1, Fscore) можно интерпретировать как взвешенное среднее precision и recall. Если F1=1, то классификатор отработал на 100% и F1=0 тогда классификатор не справился с задачей.



$$ROC = \frac{P(x|positive)}{P(x|negative)}$$

Рассчитывает отношение числа правильно распознанных случаев к числу не правильных. Процесс расчета таков: берутся данные, последовательно, и в них вычисляется это отношение. В какой то момент отношение становится константой.

AUC – интеграл под кривой.

СПАСИБО ЗА ВНИМАНИЕ!